Academy Hub — Sesión 10

Redis: clave-valor, TTL, cache, presencia de usuarios y preparación para Channels y Celery

---

Índice

1. Objetivo general
2. Qué vamos a aprender
3. Teoría: qué es Redis
4. Teoría: Redis frente a PostgreSQL
5. Instalación y arranque
6. Primeros comandos
7. Convención de claves
8. TTL
9. Cliente Python `redis-py`
10. Configuración de Django con Redis
11. Cache de datos: cache-aside
12. Cache de cursos populares
13. Invalidación de cache
14. Cache de estadísticas y agregados
15. Contadores rápidos
16. Presencia online
17. Last seen
18. Middleware de presencia
19. Rate limiting
20. Cache de vistas vs cache de datos
21. Preparación para Channels y Celery
22. Tests
23. Estructura final
24. Checklist
25. Registro de sesión
26. Commit recomendado

---

1. Objetivo general

Hasta ahora Academy Hub tiene:

```
Usuarios → Academias → Cursos → Convocatorias
    ↓
Inscripciones → Pagos → Evaluaciones → Pasaporte
    ↓
PostgreSQL
    ↓
API REST (DRF)
```

En esta sesión añadimos **Redis** como pieza de infraestructura transversal:

```
Django
  ├── PostgreSQL    (datos permanentes)
  └── Redis         (caché, TTL, presencia, contadores, rate limiting)
```

Redis no sustituye a PostgreSQL. Los dos tienen responsabilidades distintas:

- **PostgreSQL**: la verdad permanente.
- **Redis**: velocidad y temporalidad.

Si Redis se cae o se borra, Academy Hub debe seguir funcionando con los datos de PostgreSQL.

Al terminar la sesión tendremos:

- Redis instalado y funcionando.
- Conexión desde Python.
- Configuración vía `.env`.
- Redis como cache de Django.
- Operaciones `GET` / `SET` / `DELETE` / `EXPIRE` / `INCR`.
- TTL aplicado a datos temporales.
- Cache de cursos populares.
- Invalidación de cache.
- Contadores rápidos (visitas, favoritos).
- Presencia online básica.
- `last_seen`.
- Middleware de presencia.
- Rate limiting sencillo.
- Preparación para Channels y Celery.

---

2. Qué vamos a aprender

- Redis
- Key-value
- In-memory
- `SET`, `GET`, `DELETE`
- `INCR`
- `EXPIRE`
- TTL
- Cache
- Cache invalidation
- Cache-aside
- Namespaces / key prefixes
- Atomic operations
- Counters
- Rate limiting
- Presence
- Last seen
- Serialization
- Redis databases
- Django cache framework
- `redis-py`
- Middleware

---

3. Teoría: qué es Redis

Redis es un sistema de almacenamiento en memoria, extremadamente rápido, que trabaja con pares **clave → valor**.

```
"user:15:online"        → "1"
"product:25:views"      → 154
"home:popular_courses"  → [5, 17, 22, 31]
```

Vive en RAM, así que responde en milisegundos. Pero **es efímero por diseño**: si se reinicia, se pierde lo que no se haya persistido explícitamente.

Características clave

- **En memoria**: rapidísimo.
- **TTL** (`EXPIRE`): cada clave puede tener fecha de caducidad.

- **Operaciones atómicas** (`INCR`): suma segura en concurrencia.
- **Estructuras**: strings, listas, sets, sorted sets, hashes.

---

4. Teoría: Redis frente a PostgreSQL

PostgreSQL (verdad permanente)

- Usuarios.
- Academias.
- Cursos.
- Convocatorias.
- Inscripciones.
- Pagos.
- Evaluaciones.
- Pasaporte.

Redis (velocidad + temporalidad)

- Cache de cursos populares.
- Contadores rápidos.
- Presencia online.
- Rate limiting.
- Broker de Celery (futuro).
- Channel layer de Channels (futuro).

Regla de oro

> PostgreSQL = verdad permanente. Redis = velocidad + temporalidad.

Si Redis se cae:

- El cache de populares se recalcula.
- La presencia online se resetea.
- Los contadores rápidos se pierden (se pueden reconstruir).

Pero **nada crítico se pierde**.

---

5. Instalación y arranque

5.1. Comprobar instalación

```
redis-server --version
redis-cli --version
```

5.2. Arrancar Redis

```
redis-server
```

Escucha por defecto en:

```
127.0.0.1:6379
```

5.3. Comprobar conexión

En otra terminal:

```
redis-cli
```

Dentro:

```
PING
```

Resultado esperado:

```
PONG
```

5.4. Nota para Windows

En Windows no hay Redis nativo. Opciones:

- **WSL2** (recomendado): instalar Ubuntu y `apt install redis-server`.
- **Memurai**: puerto de Redis para Windows.
- **Docker**: `docker run -d -p 6379:6379 redis:7`.

Cualquiera de las tres funciona. El resto de la sesión asume que Redis está escuchando en `127.0.0.1:6379`.

---

6. Primeros comandos

Dentro de `redis-cli`:

```
SET academyhub "online"
GET academyhub
DEL academyhub
GET academyhub
```

Resultados esperados:

```
OK
"online"
(integer) 1
(nil)
```

Contador atómico

```
SET academyhub:test 10
INCR academyhub:test
```

Resultado:

```
(integer) 11
```

TTL

```
SET user:5:online 1 EX 60
TTL user:5:online
```

Resultado:

```
(integer) 47
```

Cuando el TTL llega a 0, la clave desaparece automáticamente.

---

7. Convención de claves

Usamos **namespaces** con `:` como separador y prefijo `academyhub:`:

```
academyhub:user:<id>:online
academyhub:user:<id>:last_seen
```

```
academyhub:course:<id>:views
academyhub:course:<id>:favorite_count
academyhub:academy:<id>:views
academyhub:home:popular_courses
academyhub:home:stats
academyhub:rate:enrollments:<user_id>
academyhub:rate:reviews:<user_id>
```

Esto evita colisiones con otras apps que compartan el mismo Redis.

---

8. TTL

TTL = **Time To Live**. Cuánto tiempo vive una clave.

Ejemplos de uso:

Dato	TTL razonable
Usuario online	60 s
Cache de populares	60 s
Cache de stats	300 s
Rate limit	60 s
Cache de categorías	600 s

Sin TTL habría que limpiar manualmente. Con TTL, **Redis limpia solo**.

---

9. Cliente Python `redis-py`

```
pip install redis
pip freeze > requirements.txt
```

`.env`

Añade al `.env` existente:

```
REDIS_URL=redis://127.0.0.1:6379/0
```

`.env.example`

Añade:

```
REDIS_URL=redis://127.0.0.1:6379/0
```

`core/redis.py`

Archivo nuevo. Cliente nativo reutilizable:

```
import os

import redis

redis_client = redis.Redis.from_url(
    os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0"),
    decode_responses=True,
)
```

`decode_responses=True` hace que los valores lleguen como `str` en lugar de `bytes`.

Probar

```
python manage.py shell
```

```
from core.redis import redis_client
redis_client.ping()
```

Resultado:

```
True
```

---

10. Configuración de Django con Redis

`config/settings/base.py`

Al final del archivo, añade:

```
# ============================================================
# REDIS / CACHE
# ============================================================

import os

REDIS_URL = os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0")

CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": REDIS_URL,
        "TIMEOUT": 300,
        "KEY_PREFIX": "academyhub",
    }
}
```

`config/settings/testing.py`

Para tests rápidos, cambia a cache en memoria:

```
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.locmem.LocMemCache",
        "LOCATION": "academyhub-testing",
    }
}
```

Así los tests no dependen de Redis. Los tests de integración con Redis real se marcarán como tales y se ejecutarán aparte.

Probar

```
python manage.py shell
```

```
from django.core.cache import cache

cache.set("hello", "Academy Hub", timeout=30)
cache.get("hello")
```

Resultado:

```
'Academy Hub'
```

Y en `redis-cli`:

```
SCAN 0
```

Debería aparecer `academyhub:hello`.

---

11. Cache de datos: cache-aside

El patrón **cache-aside**:

```
Petición → ¿está en Redis? → SÍ → devolver
                           → NO → PostgreSQL → guardar en Redis → devolver
```

Se aplica a datos:

- costosos de calcular,
- repetidos muchas veces,
- que toleran estar desactualizados unos segundos.

En Academy Hub, los candidatos naturales:

- Cursos populares (portada).
- Stats globales (dashboard).
- Categorías activas (menú).
- Academias destacadas.

---

12. Cache de cursos populares

Archivo nuevo: `courses/cache.py`

```python
from django.core.cache import cache
from django.db.models import Count

from courses.models import Course, CourseStatus

POPULAR_COURSES_KEY = "home:popular_courses"
POPULAR_COURSES_TIMEOUT = 60


def get_popular_courses():
    course_ids = cache.get(POPULAR_COURSES_KEY)

    if course_ids is None:
```

```
        course_ids = list(
            Course.objects
            .filter(status=CourseStatus.PUBLISHED)
            .annotate(favorite_count=Count("favorites"))
            .order_by("-favorite_count", "-views_count")
            .values_list("id", flat=True)[:10]
        )
        cache.set(
            POPULAR_COURSES_KEY,
            course_ids,
            timeout=POPULAR_COURSES_TIMEOUT,
        )

    if not course_ids:
        return []

    courses = (
        Course.objects
        .filter(id__in=course_ids)
        .select_related("academy", "category")
        .prefetch_related("images")
    )

    order = {course_id: i for i, course_id in enumerate(course_ids)}
    return sorted(courses, key=lambda c: order[c.id])
```

Por qué guardamos IDs y no objetos:

- Son ligeros.
- No dependen de serialización ORM.
- Siempre devolvemos datos frescos de PostgreSQL.

Usar en la vista

Archivo: `core/views.py`

```
from courses.cache import get_popular_courses


def home(request):
    popular_courses = get_popular_courses()
    return render(
        request,
        "core/home.html",
        {"popular_courses": popular_courses},
    )
```

Template

Archivo: `templates/core/home.html` (añadir al final del `{% block content %}`)

```
<h2>Cursos populares</h2>

{% for course in popular_courses %}
    <article>
        <h3>
            <a href="{{ course.get_absolute_url }}">{{ course.title }}</a>
        </h3>
        <p>{{ course.academy.name }}</p>
    </article>
{% empty %}
    <p>No hay cursos populares todavía.</p>
{% endfor %}
```

---

13. Invalidación de cache

Cuando los favoritos cambian, el cache de populares queda obsoleto. Hay que invalidarlo.

Añadir a `courses/cache.py`:

```
def invalidate_popular_courses():
    cache.delete(POPULAR_COURSES_KEY)
```

Usar en `courses/views.py` después de crear/eliminar un favorito:

```
from courses.cache import invalidate_popular_courses

@login_required
@require_POST
def toggle_course_favorite(request, pk):
    # ... lógica existente ...
    invalidate_popular_courses()
    return redirect(next_url)
```

Y lo mismo en `toggle_academy_favorite` si decides cachear academias populares.

Estrategia

Combinamos:

- **TTL** (60 s) como red de seguridad.
- **Invalidación explícita** cuando ocurre un evento relevante.

14. Cache de estadísticas y agregados

Las stats globales del dashboard se calculan con `aggregate()`. Son costosas y cambian poco. Perfectas para cachear.

Archivo nuevo: `dashboard/cache.py`

```
from django.core.cache import cache
from django.db.models import Avg

from courses.models import Convocatoria, Course
from enrollments.models import Enrollment
from reviews.models import CourseReview

STATS_KEY = "home:stats"
STATS_TIMEOUT = 300


def get_global_stats():
    stats = cache.get(STATS_KEY)

    if stats is None:
        stats = {
            "total_courses": Course.objects.count(),
            "published_courses": Course.objects.filter(
                status="published"
            ).count(),
            "total_enrollments": Enrollment.objects.count(),
            "total_reviews": CourseReview.objects.count(),
            "avg_rating": float(
                CourseReview.objects.aggregate(avg=Avg("rating"))["avg"] or 0
            ),
            "avg_price": float(
                Convocatoria.objects.filter(is_active=True).aggregate(
                    avg=Avg("price")
                )["avg"] or 0
            ),
        }
        cache.set(STATS_KEY, stats, timeout=STATS_TIMEOUT)

    return stats


def invalidate_global_stats():
    cache.delete(STATS_KEY)
```

Usar en `dashboard/views.py`**:**

```
from dashboard.cache import get_global_stats

def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["stats"] = get_global_stats()
    return context
```

Invalidar en `EnrollmentCreateView`, `CourseCreateView`, `CourseReviewCreateView`. Es una línea por vista.

---

15. Contadores rápidos

Los contadores de visitas se pueden incrementar en Redis con `INCR` (atómico y rapidísimo) y sincronizar periódicamente con PostgreSQL.

Archivo nuevo: `courses/redis_services.py`

```
from core.redis import redis_client


def increment_course_views(course_id):
    key = f"academyhub:course:{course_id}:views"
    return redis_client.incr(key)


def get_course_views(course_id):
    key = f"academyhub:course:{course_id}:views"
    value = redis_client.get(key)
    return int(value) if value else 0


def increment_academy_views(academy_id):
    key = f"academyhub:academy:{academy_id}:views"
    return redis_client.incr(key)


def get_academy_views(academy_id):
    key = f"academyhub:academy:{academy_id}:views"
    value = redis_client.get(key)
    return int(value) if value else 0
```

Decisión de arquitectura

En la Sesión 6 ya tenemos `Course.views_count` en PostgreSQL. Mantenemos:

- **PostgreSQL**: contador permanente (el real).
- **Redis**: contador rápido para mostrar en portada.

Más adelante, Celery sincronizará Redis → PostgreSQL cada X minutos.

Usar en `CourseDetailView`

```
from courses.redis_services import increment_course_views

def get_object(self, queryset=None):
    # ... lógica existente ...
    increment_course_views(course.pk)
    return course
```

---

16. Presencia online

Archivo nuevo: `accounts/presence.py`

```
from datetime import datetime, timezone

from core.redis import redis_client

ONLINE_TIMEOUT = 60


def mark_user_online(user_id):
    key = f"academyhub:user:{user_id}:online"
    redis_client.set(key, "1", ex=ONLINE_TIMEOUT)


def is_user_online(user_id):
    key = f"academyhub:user:{user_id}:online"
    return redis_client.exists(key) == 1


def update_last_seen(user_id):
    key = f"academyhub:user:{user_id}:last_seen"
    redis_client.set(key, datetime.now(timezone.utc).isoformat())


def get_last_seen(user_id):
    key = f"academyhub:user:{user_id}:last_seen"
    return redis_client.get(key)
```

Cómo funciona:

- Cada vez que el usuario hace una petición, `mark_user_online` renueva el TTL de 60 s.
- Si pasa 60 s sin actividad, la clave expira y el usuario aparece como offline.
- No ensuciamos PostgreSQL con un campo `is_online` que cambiaría constantemente.

---

17. Last seen

`last_seen` se guarda como string ISO 8601 en Redis. Es un dato que **no expira** por defecto, pero que se puede migrar a PostgreSQL más adelante si queremos persistencia real.

Si Redis se cae, se pierde el `last_seen` actual. Es aceptable para un dato de actividad.

Más adelante, en la sesión de Celery, se puede añadir una tarea periódica que copie el `last_seen` de Redis a un campo `User.last_seen` en PostgreSQL.

---

18. Middleware de presencia

Archivo nuevo: `accounts/middleware.py`

```
from .presence import mark_user_online, update_last_seen


class UserPresenceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.user.is_authenticated:
            try:
                mark_user_online(request.user.pk)
                update_last_seen(request.user.pk)
            except Exception:
                # Redis caído: no rompemos la petición
                pass

        return self.get_response(request)
```

Registrar en `config/settings/base.py`, **después** de `AuthenticationMiddleware`:

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "accounts.middleware.UserPresenceMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

Importante: el `try/except` garantiza que si Redis se cae, Academy Hub **sigue funcionando**. Esa es la regla de oro.

Mostrar online en el perfil público

En `accounts/views.py`, en `UserPublicProfileView.get_context_data`:

```
from accounts.presence import is_user_online, get_last_seen

def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["is_online"] = is_user_online(self.object.pk)
    context["last_seen"] = get_last_seen(self.object.pk)
    return context
```

En `templates/accounts/public_profile.html`:

```
{% if is_online %}
    <span class="badge online">Online</span>
{% else %}
    <span class="badge offline">Offline</span>
    {% if last_seen %}
        <small>Última conexión: {{ last_seen }}</small>
    {% endif %}
{% endif %}
```

---

19. Rate limiting

Archivo nuevo: `core/rate_limit.py`

```
from core.redis import redis_client
```

```
def check_rate_limit(key, limit, window):
    """
    Devuelve True si la petición está permitida.
    Ventana fija: cada `window` segundos se resetea el contador.
    """
    current = redis_client.incr(key)

    if current == 1:
        redis_client.expire(key, window)

    return current <= limit
```

Cómo funciona:

- `INCR` suma 1 y devuelve el total.
- Si es la primera vez, se aplica `EXPIRE window`.
- Si supera `limit`, devuelve `False`.

Ejemplo: limitar inscripciones

En `enrollments/views.py`, en `EnrollmentCreateView.form_valid`, **antes** de crear la inscripción:

```
from core.rate_limit import check_rate_limit

rate_key = f"academyhub:rate:enrollments:{self.request.user.pk}"

if not check_rate_limit(rate_key, limit=5, window=60):
    messages.error(
        self.request,
        "Has realizado demasiadas inscripciones. Inténtalo más tarde.",
    )
    return self.form_invalid(form)
```

Ejemplo: limitar reseñas

En `reviews/views.py`, en `CourseReviewCreateView.form_valid`:

```
from core.rate_limit import check_rate_limit

rate_key = f"academyhub:rate:reviews:{self.request.user.pk}"

if not check_rate_limit(rate_key, limit=10, window=60):
    messages.error(
        self.request,
        "Has enviado demasiadas reseñas. Inténtalo más tarde.",
```

```
    )
    return self.form_invalid(form)
```

Nota

Esto es **fixed window**. Funciona bien para el 90% de los casos. Para casos más avanzados (sliding window, token bucket) se usaría Redis más sofisticado o librerías como `django-ratelimit`. Para esta sesión, es suficiente.

---

20. Cache de vistas vs cache de datos

Django permite cachear la respuesta completa de una vista con `@cache_page(60)`. **No lo vamos a usar en la home** porque la home depende del usuario autenticado (favoritos, mensajes, etc.). Cachear la página entera daría respuestas incorrectas.

Cacheamos **datos**, no respuestas. Es la decisión correcta para una plataforma personalizada.

---

21. Preparación para Channels y Celery

Redis ya está configurado. En sesiones futuras:

Channels (Sesión 11)

```
WebSocket → Channels → Redis (channel layer) → Consumer
```

Redis distribuirá los mensajes entre procesos. La `REDIS_URL` ya está lista.

Celery (Sesión 13)

```
Django → Celery task → Redis (broker) → Worker
```

Redis actuará como broker de tareas. La misma `REDIS_URL` sirve.

Convención: separar bases lógicas

En `.env` de producción podemos separar:

```
REDIS_URL=redis://127.0.0.1:6379/0          # cache
CELERY_BROKER_URL=redis://127.0.0.1:6379/1  # broker
CHANNELS_REDIS_URL=redis://127.0.0.1:6379/2 # channel layer
```

En desarrollo, con una sola base `/0`, es suficiente.

---

22. Tests

22.1. `courses/tests/test_cache.py`

Archivo nuevo.

```
from django.core.cache import cache
from django.test import TestCase

from courses.cache import (
    POPULAR_COURSES_KEY,
    get_popular_courses,
    invalidate_popular_courses,
)


class PopularCoursesCacheTests(TestCase):

    def setUp(self):
        cache.clear()

    def tearDown(self):
        cache.clear()

    def test_cache_can_store_value(self):
        cache.set("test:key", "value", timeout=30)
        self.assertEqual(cache.get("test:key"), "value")

    def test_popular_cache_can_be_invalidated(self):
        cache.set(POPULAR_COURSES_KEY, [1, 2, 3], timeout=60)
        invalidate_popular_courses()
        self.assertIsNone(cache.get(POPULAR_COURSES_KEY))
```

22.2. `accounts/tests/test_presence.py`

Archivo nuevo.

```
from django.test import TestCase

from accounts.presence import (
    is_user_online,
    mark_user_online,
    update_last_seen,
    get_last_seen,
)
from django.contrib.auth import get_user_model

User = get_user_model()


class PresenceTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
        )

    def test_mark_user_online(self):
        mark_user_online(self.user.pk)
        self.assertTrue(is_user_online(self.user.pk))

    def test_last_seen_can_be_set(self):
        update_last_seen(self.user.pk)
        self.assertIsNotNone(get_last_seen(self.user.pk))
```

Nota: estos tests usan el cache configurado. En `testing.py` está `LocMemCache`, así que funcionan sin Redis.

22.3. `core/tests/test_rate_limit.py`

Archivo nuevo.

```
from django.core.cache import cache
from django.test import TestCase

from core.rate_limit import check_rate_limit


class RateLimitTests(TestCase):

    def setUp(self):
        cache.clear()

    def tearDown(self):
```

```
        cache.clear()

    def test_allows_under_limit(self):
        key = "test:rate:1"
        for _ in range(3):
            self.assertTrue(check_rate_limit(key, limit=5, window=60))

    def test_blocks_over_limit(self):
        key = "test:rate:2"
        for _ in range(5):
            check_rate_limit(key, limit=5, window=60)

        self.assertFalse(check_rate_limit(key, limit=5, window=60))
```

22.4. Ejecutar

```
python manage.py test
```

Todos los tests deben pasar. Los de Redis usan `LocMemCache` en testing, así que no dependen del servidor real.

---

23. Estructura final

```
academy_hub/
    core/
        redis.py              ← NUEVO
        rate_limit.py         ← NUEVO
    accounts/
        presence.py           ← NUEVO
        middleware.py         ← NUEVO
    courses/
        cache.py              ← NUEVO
        redis_services.py     ← NUEVO
    dashboard/
        cache.py              ← NUEVO
    config/
        settings/
            base.py           ← MODIFICADO
            testing.py        ← MODIFICADO
    .env                      ← MODIFICADO
    .env.example              ← MODIFICADO
    requirements.txt          ← MODIFICADO
```

24. Checklist

Redis

- [x] Redis instalado.
- [x] `redis-server` funcionando.
- [x] `redis-cli PING` → `PONG`.
- [x] `SET` / `GET` / `DEL`.
- [x] `TTL` / `EXPIRE`.
- [x] `INCR`.

Python

- [x] `pip install redis`.
- [x] `REDIS_URL` en `.env` y `.env.example`.
- [x] `core/redis.py`.

Django cache

- [x] Backend `RedisCache`.
- [x] `TIMEOUT = 300`.
- [x] `KEY_PREFIX = "academyhub"`.
- [x] `cache.set` / `cache.get` / `cache.delete`.

Marketplace

- [x] Cursos populares cacheados.
- [x] Invalidación al cambiar favoritos.
- [x] Cache-aside.

Contadores

- [x] `INCR` para visitas de cursos y academias.
- [x] Redis para rápido, PostgreSQL para permanente.

Presencia

- [x] `mark_user_online`.
- [x] `is_user_online`.
- [x] `update_last_seen`.
- [x] Middleware `UserPresenceMiddleware`.

Rate limit

- [x] `check_rate_limit`.
- [x] Aplicado a inscripciones y reseñas.

Tests

- [x] Cache set/get.
- [x] Invalidación.
- [x] Presencia.
- [x] Rate limiting.

---

25. Registro de sesión

```
PROYECTO: Academy Hub
SESIÓN: 10
FASE: Comunicación e infraestructura
TEMA: Redis, cache, TTL, presencia y contadores

OBJETIVO:
Integrar Redis en Academy Hub para datos rápidos, temporales y efímeros.

TEORÍA:
- Redis
- Key-value
- In-memory
- Cache
- TTL
- Cache hit / miss
- Cache invalidation
- Cache-aside
- INCR
- Operaciones atómicas
- Rate limiting
- Presence
- Middleware

TECNOLOGÍAS:
- Redis
- redis-py
- Django Cache
```

- PostgreSQL
- Django

FUNCIONALIDADES:
- Redis conectado
- Cache Django
- Cursos populares cacheados
- Invalidación de cache
- Contadores rápidos
- Presencia online
- Last seen
- Middleware de presencia
- Rate limit
- Preparación para Channels y Celery

ARCHIVOS CREADOS:
core/redis.py
core/rate_limit.py
accounts/presence.py
accounts/middleware.py
courses/cache.py
courses/redis_services.py
dashboard/cache.py
courses/tests/test_cache.py
accounts/tests/test_presence.py
core/tests/test_rate_limit.py

ARCHIVOS MODIFICADOS:
.env
.env.example
requirements.txt
config/settings/base.py
config/settings/testing.py
core/views.py
courses/views.py
accounts/views.py
dashboard/views.py

TESTS:
- cache set/get
- invalidación
- presencia
- rate limiting

```
COMMIT:
feat: add Redis caching presence counters and rate limiting

ESTADO:
[ ] Pendiente
[ ] En progreso
[ ] Completada

SIGUIENTE SESIÓN:
Chat en tiempo real con ASGI, Django Channels, WebSockets, Consumers,
Groups, Redis Channel Layer y mensajes en tiempo real.
```

---

26. Commit recomendado

```
git status
git add .
git commit -m "feat: add Redis caching presence counters and rate limiting"
```

Resultado final

Antes:

```
Django
    ↓
PostgreSQL
```

Ahora:

```
Django
  ├── PostgreSQL    (verdad permanente)
  └── Redis
        ├── cache
        ├── TTL
        ├── presencia
        ├── contadores
        └── rate limiting
```

La regla que se queda grabada:

> **PostgreSQL** = verdad permanente. **Redis** = velocidad + temporalidad.

Redis se convierte en pieza transversal de Academy Hub. En la Sesión 11 lo reutilizaremos para el chat en tiempo real con Channels.